

UK PATENT APPLICATION

SYSTEM AND METHOD FOR AUTONOMOUS CROSS-DOMAIN CAUSAL DISCOVERY AND BUSINESS GENERATION USING ENSEMBLE PREDICTION AND TEST-DRIVEN DEVELOPMENT

FIGURES

The following simple flow diagrams describe the content of each figure for formal patent drawing conversion. All figures use standard patent drawing conventions: rectangular boxes for processes, diamond shapes for decisions, cylinders for databases, and directional arrows for data flow.

FIGURE 1: System Architecture Overview
Innovation (4) - Closed-Loop Autonomous Business Pipeline
Claims: 1, 11, 16

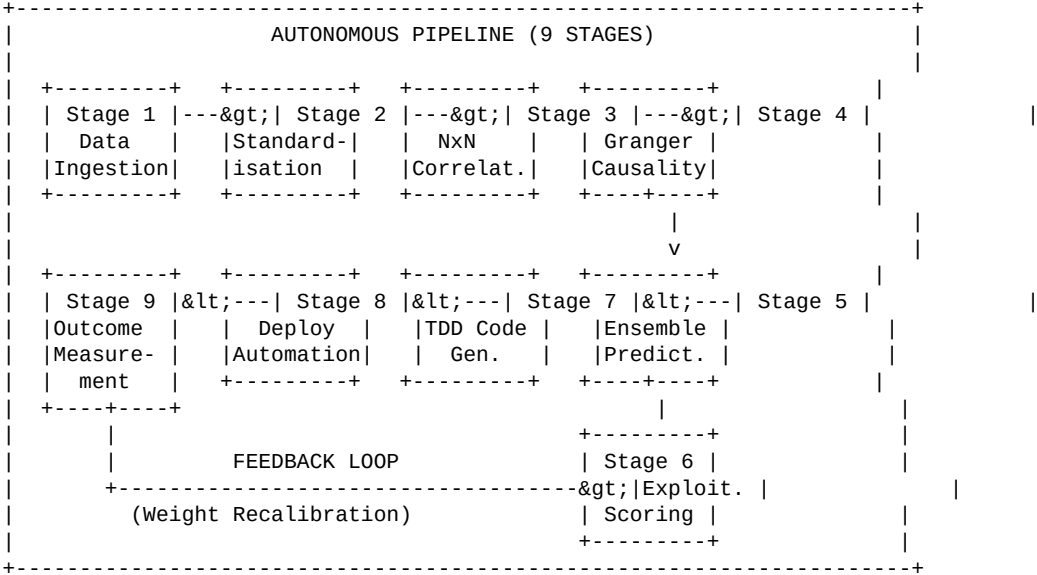


FIGURE 2: Multi-Source Data Ingestion Pipeline
Innovation (1) - Variable-Level Cross-Domain Causal Discovery
Claims: 1, 2, 11, 42, 43

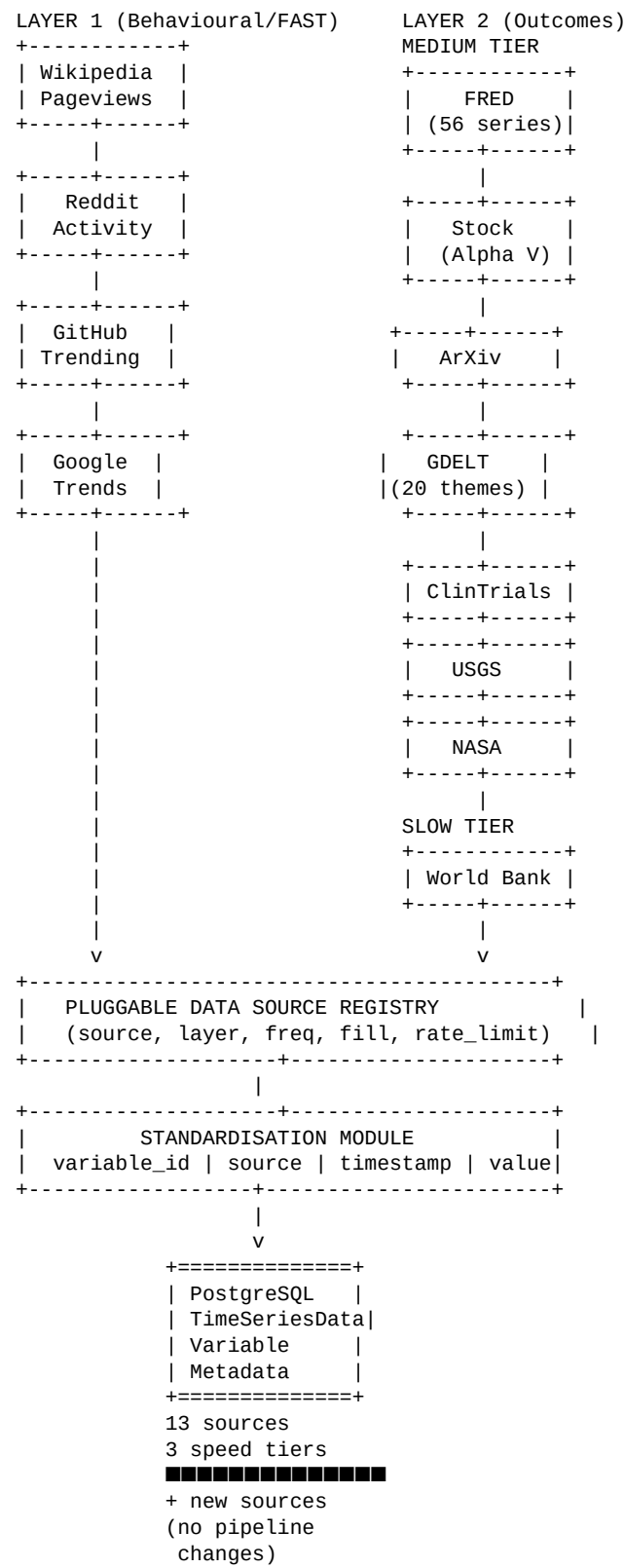


FIGURE 3: Cross-Domain Correlation Engine
Innovations (1) and (5)

Claims: 1, 3, 11

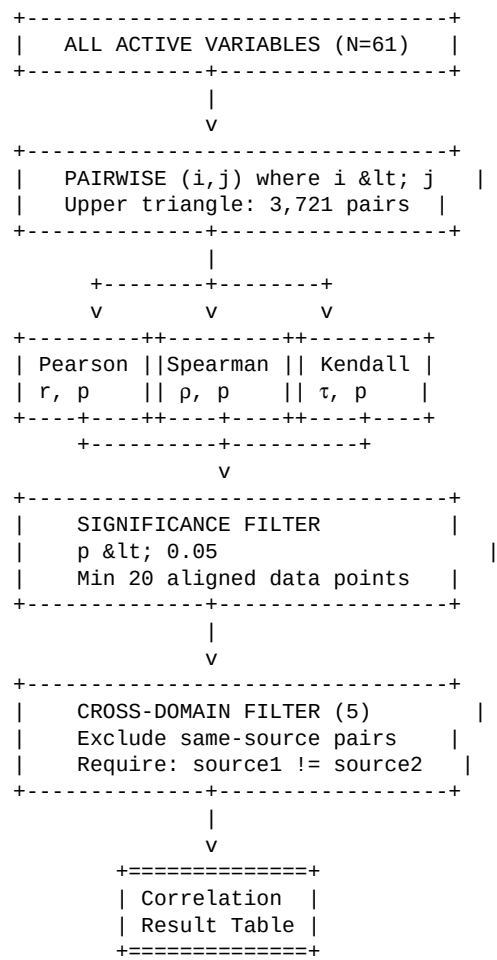


FIGURE 4: Bidirectional Granger Causality System

Innovation (1)

Claims: 1, 4, 11, 34

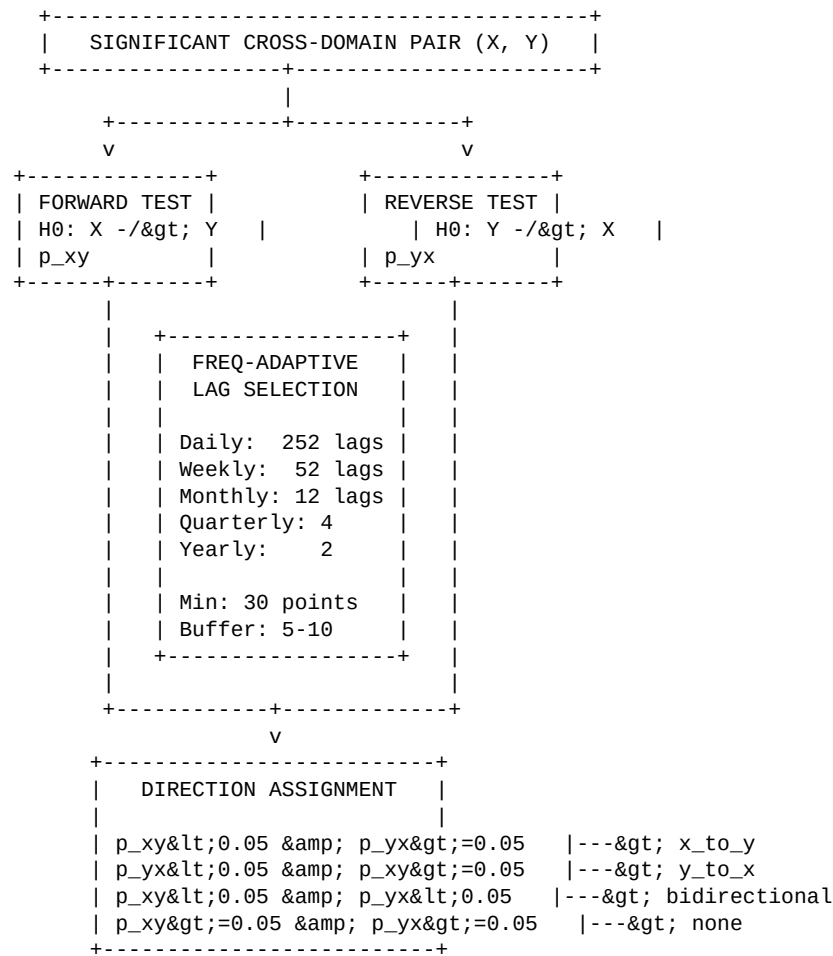


FIGURE 5: Extensible Multi-Model Ensemble Architecture

Innovation (2)

Claims: 1, 5, 6, 11, 12, 21, 24, 44

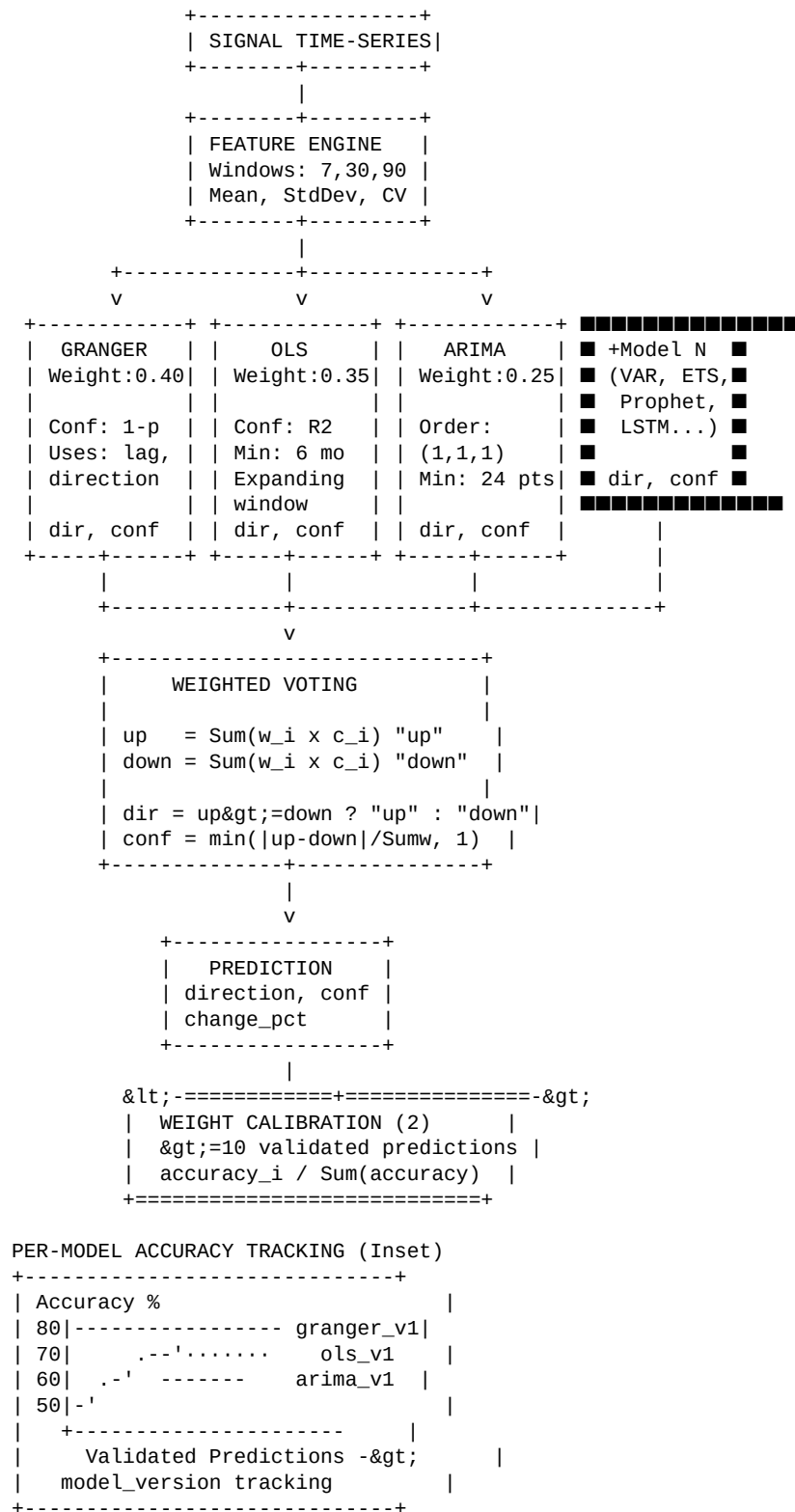


FIGURE 6: Cross-Layer Enforcement

Innovation (5)

Claims: 1, 7, 11, 20

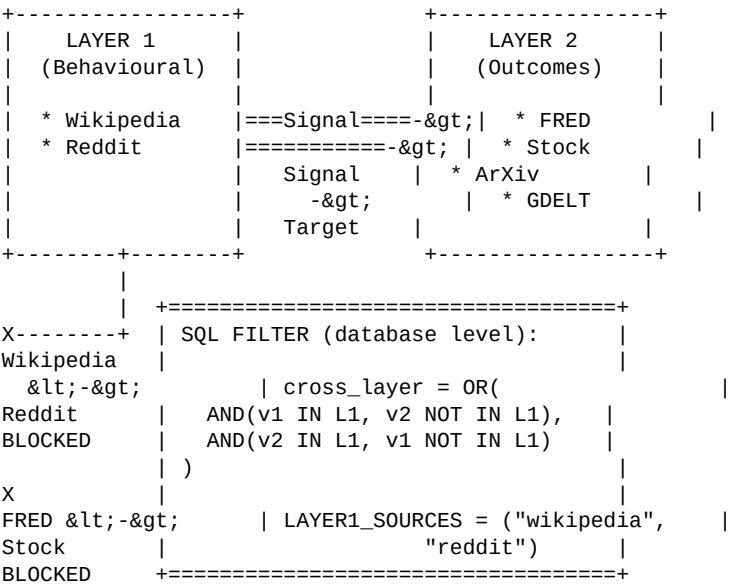


FIGURE 7: Exploitation Scoring Pipeline

Innovations (2) and (4)

Claims: 1, 8, 11, 17, 26

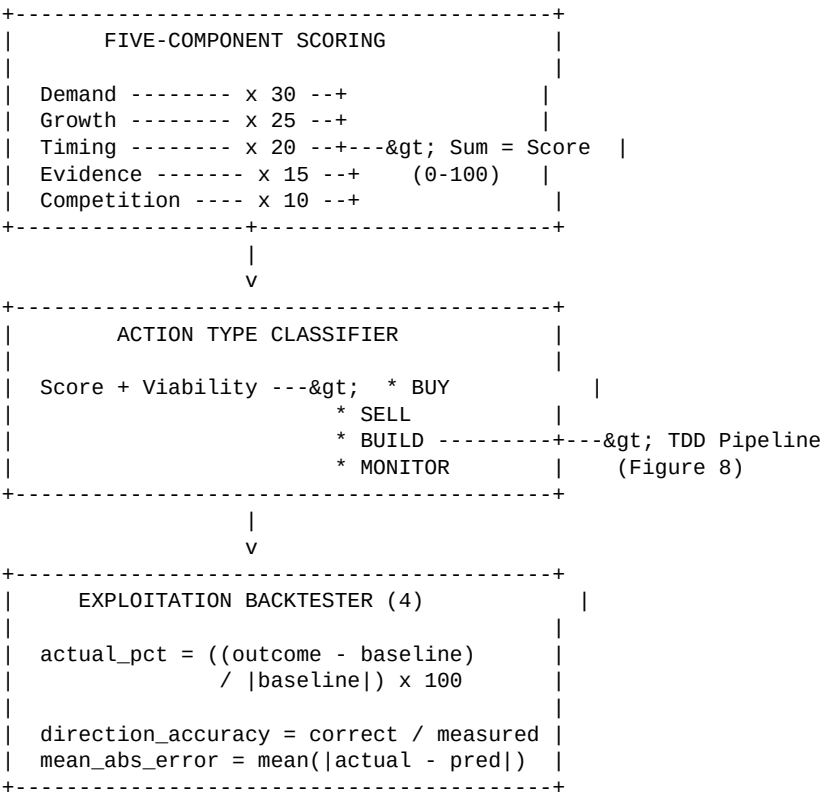


FIGURE 8: Autonomous TDD Code Generation Overview

Innovation (3)

Claims: 1, 9, 11, 13, 16, 28

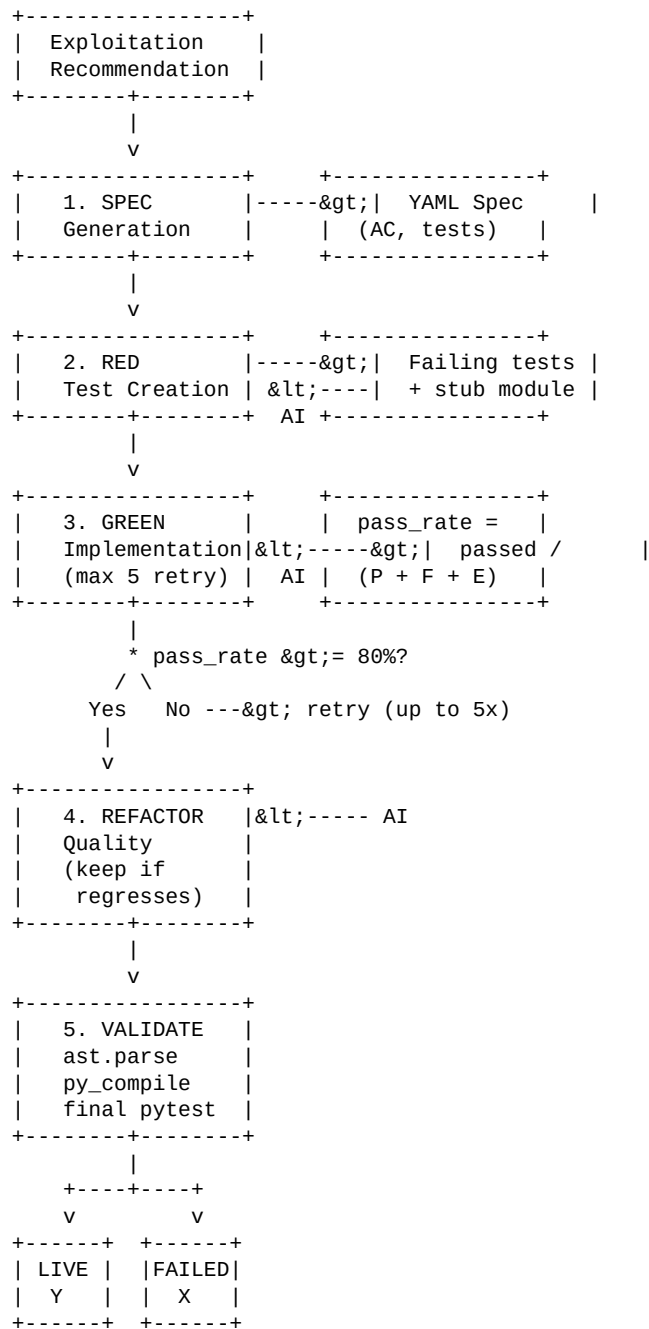


FIGURE 9: Five-Phase TDD Pipeline Detail

Innovation (3)

Claims: 9, 10, 13, 14, 22, 28, 45

SPEC PHASE +-----+ Keyword Analysis 8 categories: frontend, backend, database, auth, deployment, payments, analytics, CRUD Template Matching: Layer align: 30pts Tag match: 15pts Framework: 10pts Desc words: 5pts (max 20) Complexity Tiers: LOW: 4 AC MEDIUM: 7 AC HIGH: 12 AC Commercial Intel. Consultation (8): * Top 5 deploys * Best configs * Confidence score +-----+ <tr><td> RED PHASE +-----+ Generate test files from YAML spec Create stub module with NotImplemented- Error for all imports Verify: all tests fail (confirmed RED) +-----+ </td></tr> <tr><td> REFACTOR PHASE +-----+ AI improves quality Regression guard: If new tests fail, keep pre-refactor version +-----+ </td></tr> <tr><td> ERROR CATEGORIES +-----+ * syntax * test * frontend * import * wiring * config * runtime +-----+ </td></tr> <tr><td> GREEN PHASE +-----+ Extract symbols: regex: from X import Y Submit to AI: * test code * symbol list * prev failure (if retry attempt) pass_rate >= 80%? Yes -> REFACTOR No -> retry (max 5) +-----+ </td></tr> <tr><td> VALIDATE PHASE +-----+ 1. ast.parse (syntax) 2. py_compile (all files) 3. Final pytest run 4. Record metrics: file_count, line_count 5. Status: LIVE or FAILED + error breakdown JSON +-----+ </td></tr> <tr><td> VERIFICATION ARTEFACTS +-----+ Syntactic Correctness (AST + bytecode compilation pass) Functional Compliance (YAML AC -> test per-criterion pass/fail status) +-----+ </td></tr>	RED PHASE +-----+ Generate test files from YAML spec Create stub module with NotImplemented- Error for all imports Verify: all tests fail (confirmed RED) +-----+	REFACTOR PHASE +-----+ AI improves quality Regression guard: If new tests fail, keep pre-refactor version +-----+	ERROR CATEGORIES +-----+ * syntax * test * frontend * import * wiring * config * runtime +-----+	GREEN PHASE +-----+ Extract symbols: regex: from X import Y Submit to AI: * test code * symbol list * prev failure (if retry attempt) pass_rate >= 80%? Yes -> REFACTOR No -> retry (max 5) +-----+	VALIDATE PHASE +-----+ 1. ast.parse (syntax) 2. py_compile (all files) 3. Final pytest run 4. Record metrics: file_count, line_count 5. Status: LIVE or FAILED + error breakdown JSON +-----+	VERIFICATION ARTEFACTS +-----+ Syntactic Correctness (AST + bytecode compilation pass) Functional Compliance (YAML AC -> test per-criterion pass/fail status) +-----+
RED PHASE +-----+ Generate test files from YAML spec Create stub module with NotImplemented- Error for all imports Verify: all tests fail (confirmed RED) +-----+						
REFACTOR PHASE +-----+ AI improves quality Regression guard: If new tests fail, keep pre-refactor version +-----+						
ERROR CATEGORIES +-----+ * syntax * test * frontend * import * wiring * config * runtime +-----+						
GREEN PHASE +-----+ Extract symbols: regex: from X import Y Submit to AI: * test code * symbol list * prev failure (if retry attempt) pass_rate >= 80%? Yes -> REFACTOR No -> retry (max 5) +-----+						
VALIDATE PHASE +-----+ 1. ast.parse (syntax) 2. py_compile (all files) 3. Final pytest run 4. Record metrics: file_count, line_count 5. Status: LIVE or FAILED + error breakdown JSON +-----+						
VERIFICATION ARTEFACTS +-----+ Syntactic Correctness (AST + bytecode compilation pass) Functional Compliance (YAML AC -> test per-criterion pass/fail status) +-----+						

FIGURE 10: Deployment Automation Flow

Innovations (3) and (4)

Claims: 1, 9, 11, 15, 16, 33

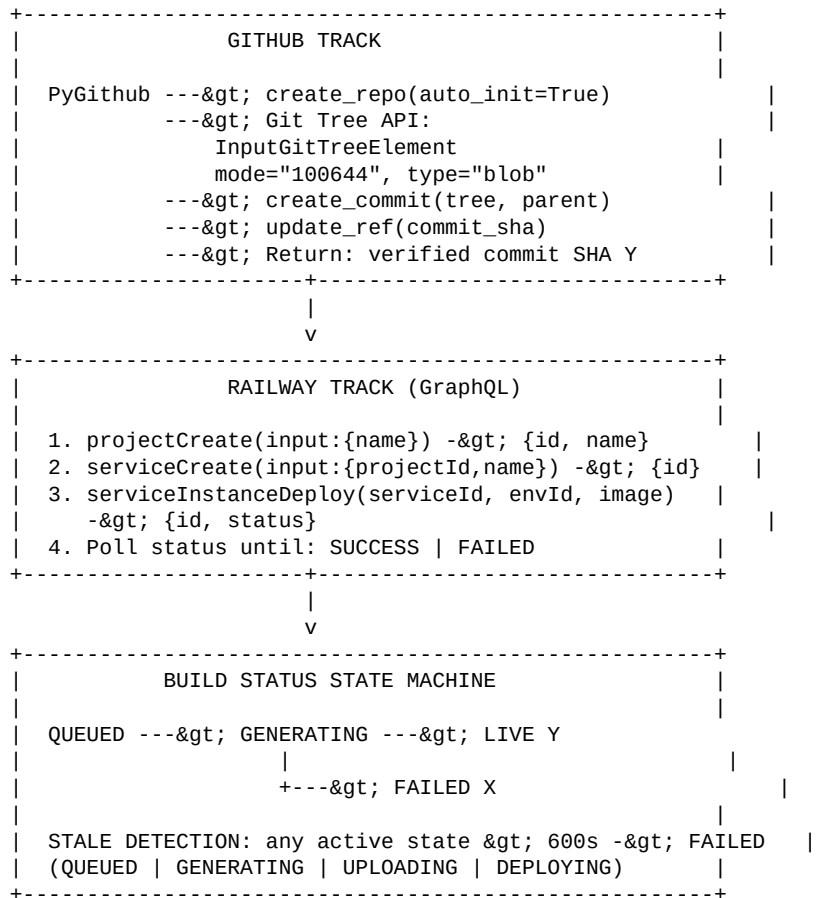


FIGURE 11: Outcome Measurement and Feedback Loop Innovations (2) and (4)

Claims: 1, 5, 6, 11, 12, 16, 17, 25

```

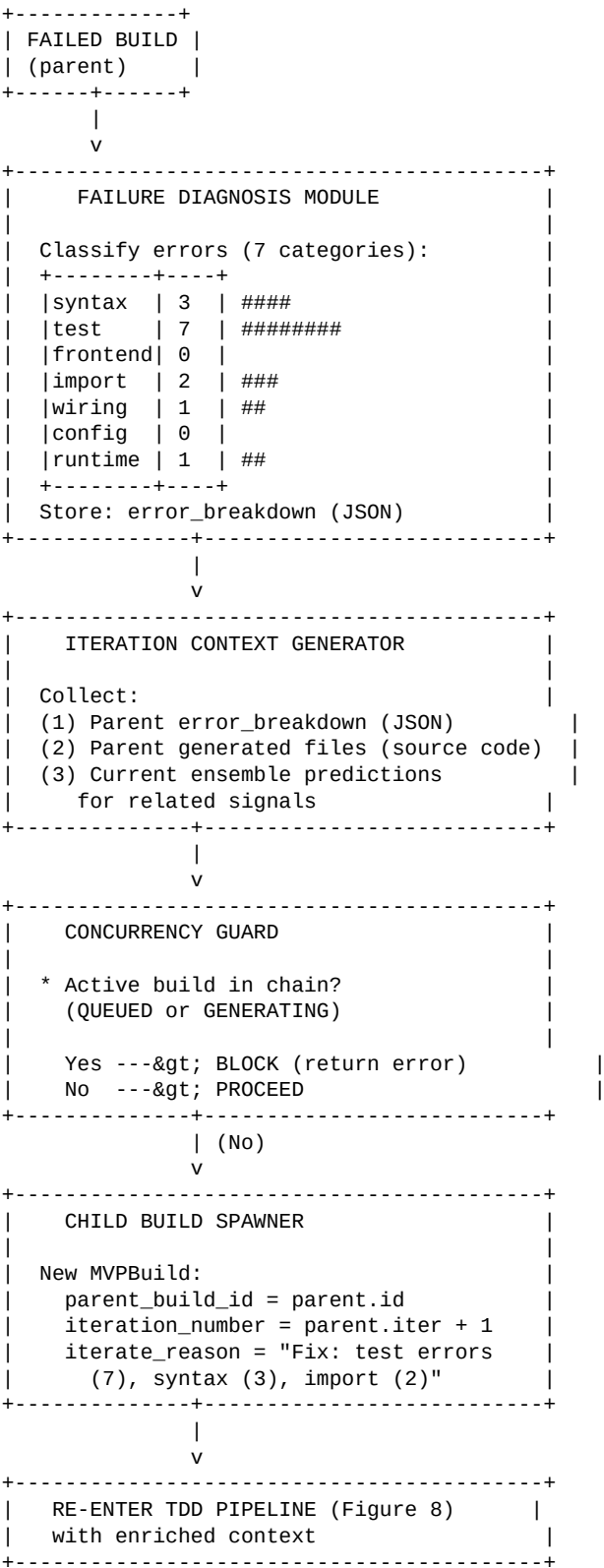
+-----+
| TRACK 1: PREDICTION VALIDATION (Daily 02:30 UTC) |
| Query: status='pending' AND target_date <= today |
| Fetch: actual value from TimeSeriesData |
| Compute: actual_direction, actual_change_pct |
| Set: direction_correct = (predicted == actual) |
| Update: status -> 'validated' |
+-----+
|
+-----+
| TRACK 2: WALK-FORWARD BACKTEST (Mon 04:00 UTC) |
| For split_idx = 6 to len(data)-1: |
|   train = data[0:split_idx] |
|   slope, intercept = polyfit(x_train, y_train, 1) |
|   predicted_y = slope x x_next + intercept |
|   direction_correct = (pred_dir == actual_dir) |
|   R2 = 1 - (ss_res / ss_tot) |
|   Store: model_version='backtest_walkforward' |
+-----+
|
+-----+
| TRACK 3: EXPLOITATION BACKTEST |
| Window: created_at -> created_at + (months x 30d) |
| actual_pct = ((outcome-baseline)/|baseline|) x 100 |
| Direction accuracy across all measured recs |
| Mean absolute error: mean(|actual - predicted|) |
+-----+
|
v
+-----+
| WEIGHT RECALIBRATION |
| Ensemble: accuracy_i / Sum(accuracy) -> new weights |
| Exploit.: discriminative components -> boost ±5pts |
|
+=====+
| Feeds back to Ensemble Engine (Figure 5) |
+=====+
+-----+

```

FIGURE 12: Build-Iterate Loop

Innovation (6)

Claims: 28, 29, 30, 31



ITERATION CHAIN TIMELINE:

+-----+	+-----+	+-----+
v1	----> v2	----> v3
FAILED	FAILED	LIVE
+-----+	+-----+	+-----+

FIGURE 13: Autonomous Scheduling Timeline

Innovation (4)

Claims: 1, 11, 16, 23, 27

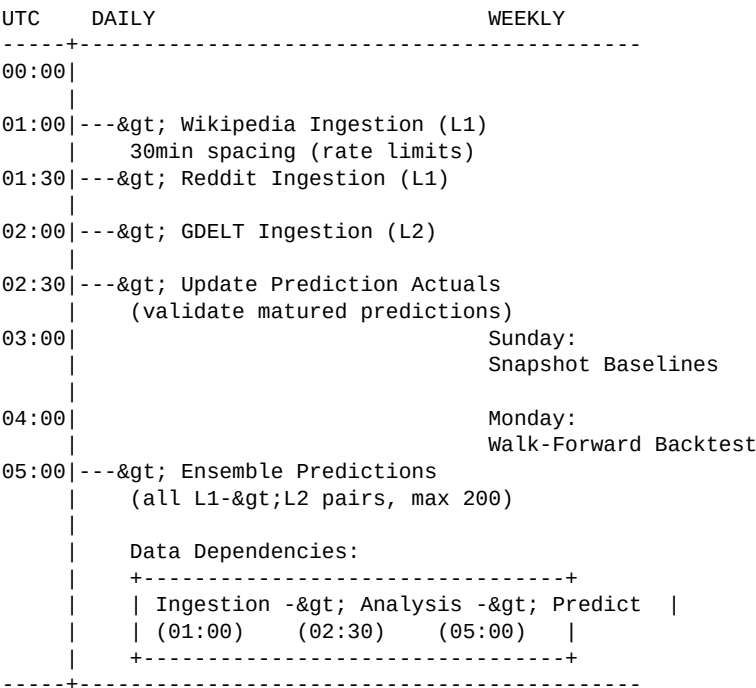
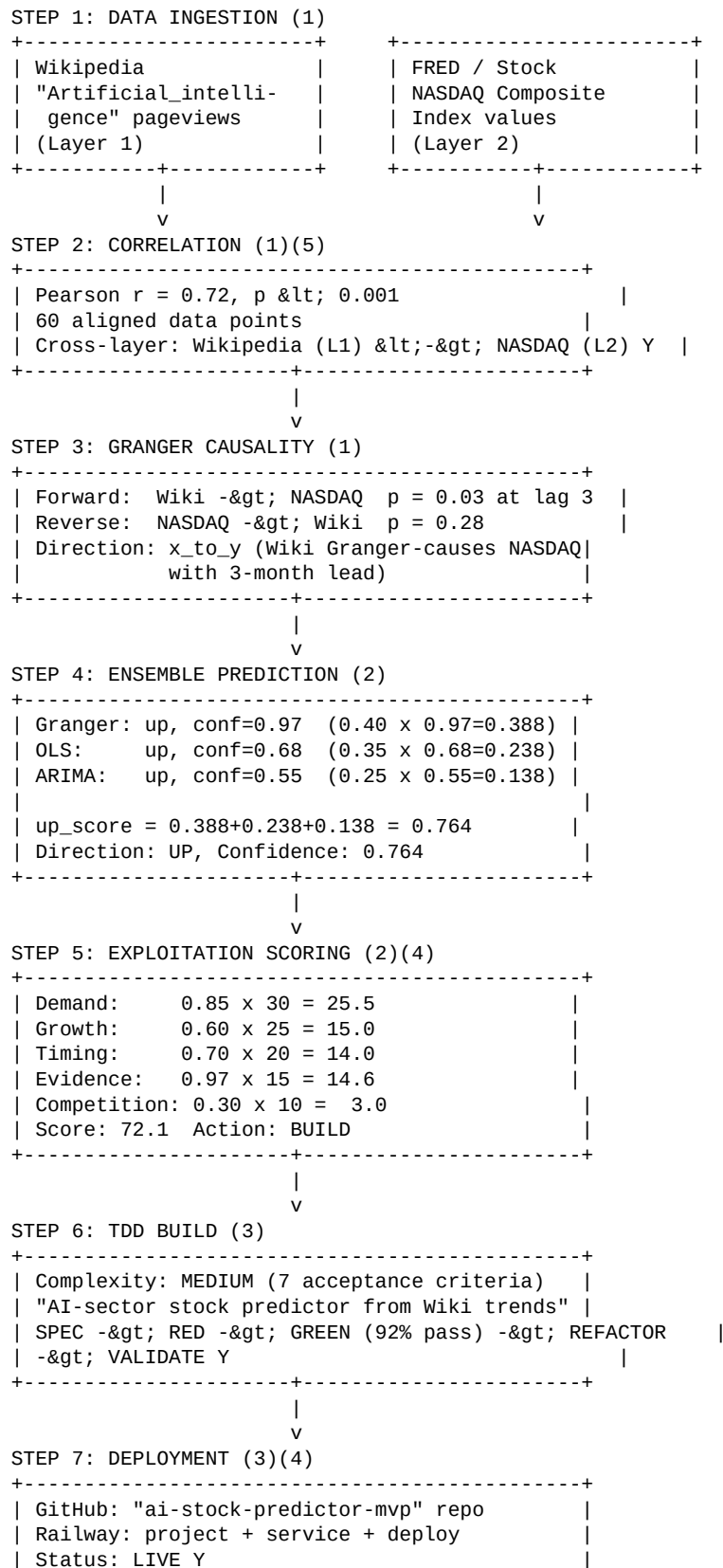


FIGURE 14: End-to-End Example Scenario

All Innovations: (1)(2)(3)(4)(5)(6)(7)

Claims: 1, 11, 16, 35

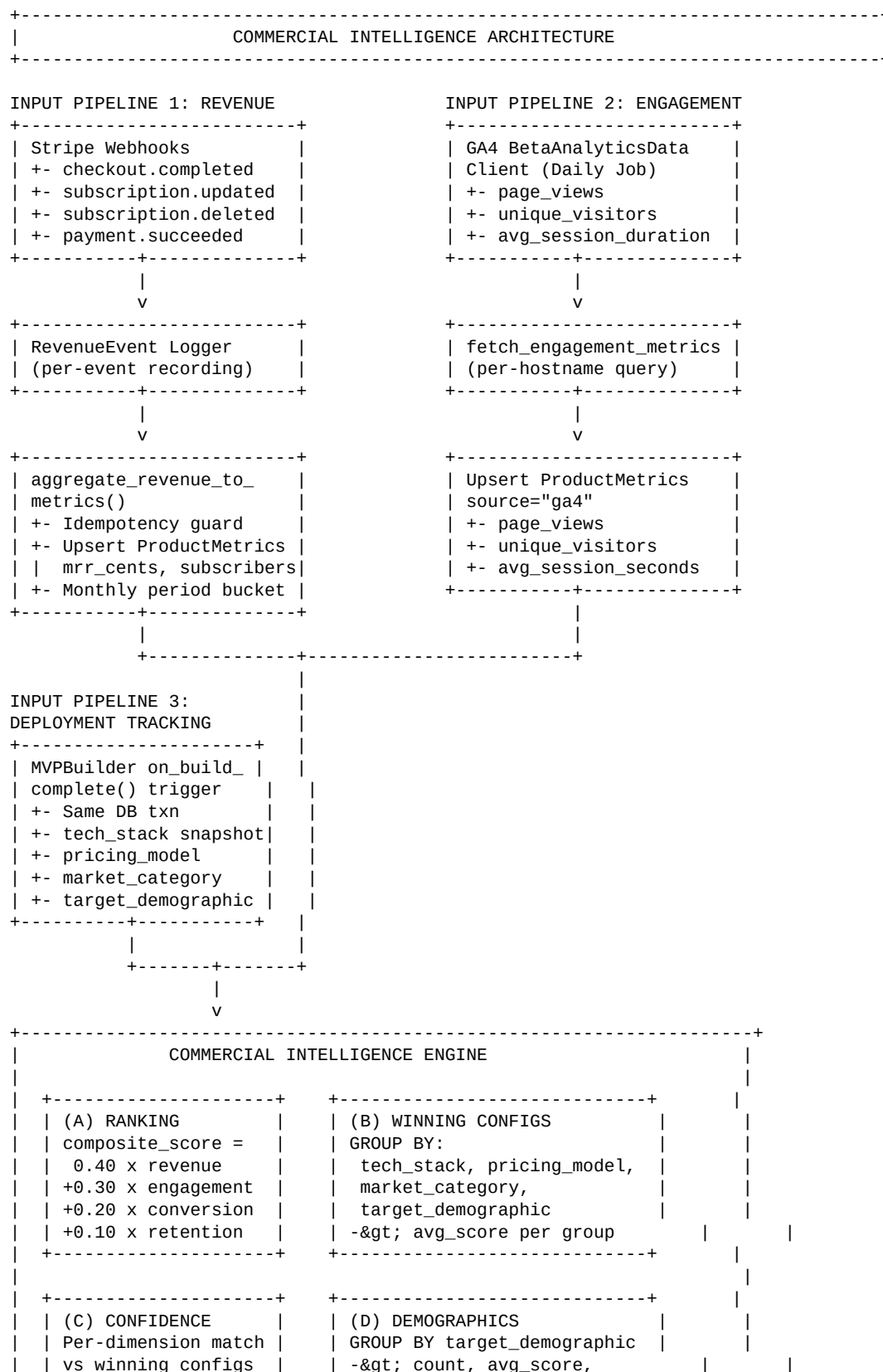


```

| URL: https://ai-stock-predictor.up.railway.app|
+-----+
|
| v
STEP 8: OUTCOME VALIDATION (2)(4)
+-----+
| +3 months (matching Granger lag 3): |
| Predicted: UP |
| Actual NASDAQ: +8.3% Direction: UP Y |
| direction_correct = True |
| Granger weight recalibrated: 0.40 -> 0.42 |
+-----+
|
| v
STEP 9: COMMERCIAL INTELLIGENCE (7)
+-----+
| Stripe webhook: 3 subscriptions -> $45 MRR |
| GA4 pull: 1,240 unique visitors, 3.2m avg |
| Composite score: 78/100 |
| Revenue: 82 x 0.40 = 32.8 |
| Engagement: 75 x 0.30 = 22.5 |
| Conversion: 68 x 0.20 = 13.6 |
| Retention: 90 x 0.10 = 9.0 |
| Winning config: React + freemium + technology |
| Confidence for next build: 82% |
| -> Injected into spec generator prompt |
+-----+

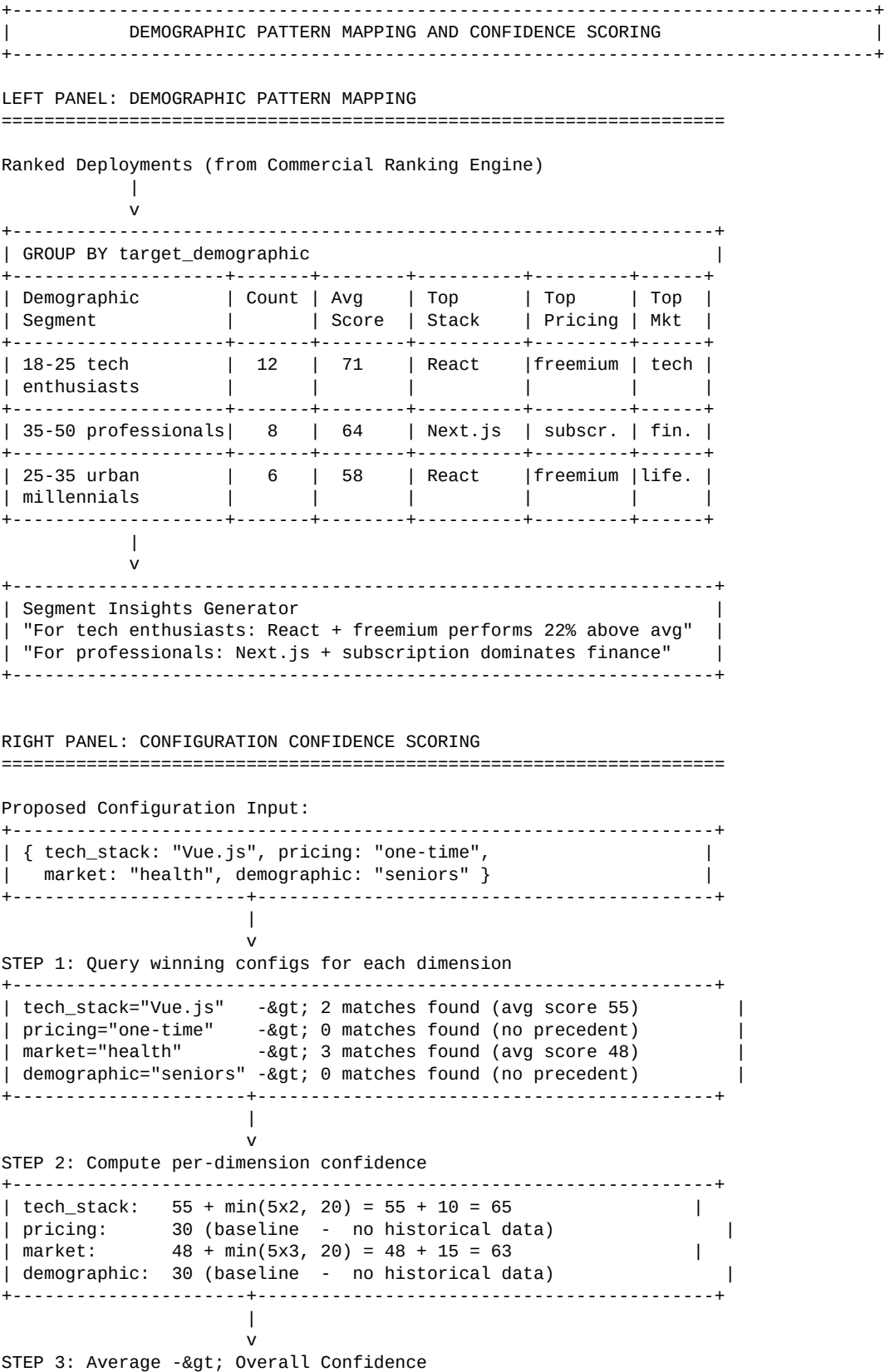
```

Claims: 35, 36, 37, 38, 39, 40



		-> 0-100 score			top_stack, top_pricing,		
		+ recommendation			top_market per segment		
	+	-----+		+	-----+		
+	+	-----+		+	-----+		+
	+	-----+		+	-----+		+
	v			v			v
+	+	-----+	+	+	-----+	+	+
	Dashboard Panel			Spec Generator			REST API
	+ Confidence %			+ Top 5 deploys			/api/commercial-
	+ Progress bar			+ Best configs			intelligence/*
	+ Recommendation			+ Market tips			7 endpoints
+	-----+		+	+ Confidence		+	-----+
				note			
			+	-----+			

FIGURE 16: Demographic Pattern Mapping and Confidence Scoring
Innovation: (7) (ML-Driven Commercial Intelligence Feedback Loop)
Claims: 38, 39, 40



```

+-----+
| overall = (65 + 30 + 63 + 30) / 4 = 47 |
+-----+

```

|
v

STEP 4: Classification

```

+-----+
| #####..... 47/100 |
| +--- RED ---+--- AMBER ---+--- GREEN ---+ |
| 0          40      70      100 |
|          ^ |
|          47 = AMBER |
| |
| Recommendation: "Moderate confidence - some supporting data, |
| but limited precedent for one-time pricing and seniors |
| demographic. Consider freemium pricing based on similar |
| market performance." |
+-----+

```